Homework 7: Projections; Regression using Linear Algebra Solutions

EECS 245, Fall 2026 at the University of Michigan

due Sunday, November 8th, 2026 at 11:59PM Ann Arbor Time

Except for the video link in Problem 1 and the notebook submission in Problem 8, write your solutions to the following problems either by writing them on a piece of paper or on a tablet and scanning your answers as a PDF. Note that you are not allowed to use LaTeX, Google Docs, or any other digital document creation software to type your answers. Homeworks are due to Pensive by 11:59PM on the due date. See the **syllabus** for details on the slip day policy.

Homework will be evaluated not only on the correctness of your answers, but on your ability to present your ideas clearly and logically. You should always explain and justify your conclusions, using sound reasoning. Your goal should be to convince the reader of your assertions. If a question does not require explanation, it will be explicitly stated.

Before proceeding, make sure you're familiar with the **collaboration policy**.

Total Points: $10 + 5 + 15 + 16 + 11 + 8 + 22 + 13 = 100$

Problem 1: Homework 6 Concept Video (10 pts)

Record a **2- to 5-minute video** answering **one** of the prompts below. These prompts are related to the ideas in **Homework 6**.

1. If you append a copy of one of a matrix's columns to that matrix, what happens to its rank and the dimension of its null space, and why?
2. Suppose a square matrix A maps two different vectors to the same vector. Can A be invertible? Explain how this relates to its null space.
3. Suppose A has more columns than rows. Explain why $A\vec{x} = \vec{b}$ cannot have exactly one solution. Can it have no solutions?
4. Can XX^T be invertible when X has more rows than columns? Explain using rank or linear independence.

The video should be **just you speaking into the camera**. Your face must be visible while you are talking for the entire video. Do not include slides, a screen recording, written work, or any other visuals.

Submit your video link to the separate assignment on Pensive called **Homework 7, Problem 1 Video Link**. You can upload your video to YouTube as unlisted, share a Zoom recording link, or share a Google Drive link; whatever works. Make sure the course staff can view the video using that link.

Your submission time for Homework 7 is the latest of your three submission times: your Problem 1 video link, your PDF, and your code.

This problem is graded on completion. Record a video of the required length and make an honest effort to explain the question, and you will receive full credit. Your explanation does not need to be polished or perfect.

Problem 2: Anonymous Feedback (5 pts)

We'd like to get your feedback on how the course has been going so far, now that we're past the halfway point and Midterm 2 is fast approaching.

You can find the survey **at this link**, which you should complete **after you've finished the rest of Homework 7**. Unlike the Homework 3 survey, **it is anonymous**, so feel free to provide candid feedback.

In order to earn the 5 points for Homework 7, Problem 2, include a screenshot of the confirmation message you see after submitting the form. (We consider it an honor code violation to include a screenshot if you didn't actually submit the form!)

Thank you for your feedback once again!

Problem 3: The Complete Solution (15 pts)

Before beginning this problem, make sure you've read both **Chapter 6.3** and **Chapter 6.4**.

a) (3 pts) Consider the matrix A and vector $\vec{b}$ defined below.

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 2 & -1 \\ 1 & -1 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 2 \end{bmatrix}$$

Find the vector $\vec{x}^*$ that minimizes $\|\vec{b} - A\vec{x}\|^2$. Show your work, but feel free to use `numpy` to compute some of the matrix operations; **here** is a video that walks through how to do this. (We're intentionally using different variables here than in the notes to have you think about the problem in general terms.)

Solution: Given

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 2 & -1 \\ 1 & -1 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 2 \end{bmatrix}$$

and $\vec{x} \in \mathbb{R}^2$, we want the vector $\vec{x}^*$ that makes the squared error $\|\vec{b} - A\vec{x}\|^2$ as small as possible.

Because A has two linearly independent columns, its columns form a 2D subspace of $\mathbb{R}^4$, and there is a *unique* least-squares solution. The standard way to get it is to use the normal equations

$$A^T A\vec{x}^* = A^T\vec{b}$$

First let's find $A^T A$:

$$A^T = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \qquad A^T A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 2 & -1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix}$$

Now we'll find the right-hand side $A^T\vec{b}$:

$$A^T\vec{b} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 9 \\ -2 \end{bmatrix}$$

So the normal equation becomes

$$\begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} x_1^* \\ x_2^* \end{bmatrix} = \begin{bmatrix} 9 \\ -2 \end{bmatrix}$$

To solve, we invert $A^T A$. The determinant is $6 \cdot 4 - (-2)(-2) = 20$, so

$$(A^T A)^{-1} = \frac{1}{20} \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

Then:

$$\vec{x}^* = (A^T A)^{-1} A^T\vec{b} = \frac{1}{20} \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 9 \\ -2 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 36 - 4 \\ 18 - 12 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 32 \\ 6 \end{bmatrix} = \begin{bmatrix} \frac{8}{5} \\ \frac{3}{10} \end{bmatrix}$$

So the least-squares minimizer is

$$\boxed{\vec{x}^* = \begin{bmatrix} \frac{8}{5} \\ \frac{3}{10} \end{bmatrix}}$$

Geometrically, $A\vec{x}^*$ is the projection of $\vec{b}$ onto the column space of A, and $\vec{x}^*$ are the coordinates of that projection in the basis given by the two columns of A.

For the remainder of the problem, we will use the same vector $\vec{b}$, but instead use the following matrix A.

$$A = \begin{bmatrix} 1 & 0 & 1 & -4 & 4 \\ 0 & 2 & 1 & -5 & 3 \\ 2 & -6 & -1 & 7 & -1 \\ 1 & -4 & -1 & 6 & -2 \end{bmatrix}$$

Note that this new matrix A has **a rank of 2**.

b) (2 pts) Now, find **one** vector $\vec{x}^*$ that minimizes $\|\vec{b} - A\vec{x}\|^2$ for this new matrix A. Again, show your work. If you've read **Chapter 6.4** closely, this should not require much calculation.

Solution: Since A has rank 2 but 5 columns, the normal equation

$$A^T A\vec{x} = A^T\vec{b}$$

do not have a unique solution, and there are infinitely many vectors $\vec{x}^*$ that minimize $\|\vec{b} - A\vec{x}\|^2$.
To find one such vector, we can keep only two linearly independent columns of A — here, we use columns 3 and 1, in that order — since removing dependent columns does not change colsp(A).

$$A' = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ -1 & 2 \\ -1 & 1 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 2 \end{bmatrix}$$

This matrix has the same two columns as in part **a)**, but in the opposite order, so the least-squares coefficients are also swapped relative to part **a)**. Therefore,

$$\vec{x}' = \begin{bmatrix} \frac{3}{10} \\ \frac{8}{5} \end{bmatrix}$$

To extend back to the full 5-dimensional $\vec{x}$, we place these coefficients in entries 3 and 1, respectively, and set the others to 0:

$$\boxed{\vec{x}^* = \begin{bmatrix} \frac{3}{10} \\ 0 \\ \frac{8}{5} \\ 0 \\ 0 \end{bmatrix}}$$

This $\vec{x}^*$ satisfies the normal equation $A^T A\vec{x} = A^T\vec{b}$ and therefore minimizes $\|\vec{b} - A\vec{x}\|^2$.

c) (4 pts) Find a basis for nullsp(A). *Hint: Try and do so efficiently, since this is the type of problem we'll see on Midterm 2.*

Solution: To find a basis for nullsp(A), we look for all vectors $\vec{x}$ that satisfy

$$A\vec{x} = \vec{0} \quad \text{where } A = \begin{bmatrix} 1 & 0 & 1 & -4 & 4 \\ 0 & 2 & 1 & -5 & 3 \\ 2 & -6 & -1 & 7 & -1 \\ 1 & -4 & -1 & 6 & -2 \end{bmatrix}$$

Since rank$(A) = 2$ and A has 5 columns, the rank-nullity theorem says

$$\dim(\text{nullsp}(A)) = 5 - 2 = 3$$

So, a basis for nullsp(A) will have 3 vectors. The easy way to find those three vectors is to determine how to write A's linearly dependent columns as linear combinations of its linearly independent columns. Columns 1 and 3 are linearly independent (they are the same two columns as in part **a)**). By inspection — that is, by guessing and checking — we can find that:

- column 2 $= -2($column 1$) + 2($column 3$)$, so

$$A \begin{bmatrix} -2 \\ -1 \\ 2 \\ 0 \\ 0 \end{bmatrix} = \vec{0}$$

- column 4 $= 1($column 1$) - 5($column 3$)$, so

$$A \begin{bmatrix} 1 \\ 0 \\ -5 \\ -1 \\ 0 \end{bmatrix} = \vec{0}$$

- column 5 $= 1($column 1$) + 3($column 3$)$, so

$$A \begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \\ -1 \end{bmatrix} = \vec{0}$$

So, a basis for nullsp(A) is given by

$$\boxed{\left\{ \begin{bmatrix} -2 \\ -1 \\ 2 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ -5 \\ -1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \\ -1 \end{bmatrix} \right\}}$$

Any linear combination of these vectors, when multiplied by A, will result in the zero vector.
This also means that

$$\text{nullsp}(A) = \text{span}\left(\left\{\begin{bmatrix}-2\\-1\\2\\0\\0\end{bmatrix}, \begin{bmatrix}1\\0\\-5\\-1\\0\end{bmatrix}, \begin{bmatrix}1\\0\\3\\0\\-1\end{bmatrix}\right\}\right)$$

d) (1 pts) Show that if $\vec{x}'$ satisfies the normal equation, $A^TA\vec{x}' = A^T\vec{b}$, and $\vec{x}_0 \in \text{nullsp}(A)$, then $\vec{x}' + \vec{x}_0$ also satisfies the normal equation. *Hint: This is two-line solution; we're mostly asking it so that you interalize **what** this means and why it's true.*

Solution: If $\vec{x}'$ satisfies the normal equation, then

$$A^TA\vec{x}' = A^T\vec{b}$$

By the definition of the null space, if $\vec{x}_0 \in \text{nullsp}(A)$, then $A\vec{x}_0 = \vec{0}$. Then:

$$A^TA(\vec{x}' + \vec{x}_0) = A^T(A\vec{x}' + A\vec{x}_0) = A^TA\vec{x}' + A^T\vec{0} = A^TA\vec{x}' + \vec{0} = A^T\vec{b}.$$

Thus, $\vec{x}'+\vec{x}_0$ also satisfies the normal equation. This means adding any vector in $\text{nullsp}(A)$ to a solution $\vec{x}'$ of the normal equation gives another valid solution.

e) (2 pts) Describe, using set notation, the complete set of vectors $\vec{x}^*$ that minimize $\|\vec{b} - A\vec{x}\|^2$. Is this set a subspace?

Solution: The complete set of vectors that minimize $\|\vec{b} - A\vec{x}\|^2$ is all vectors that result from starting with a particular solution $\vec{x}'$ and adding any vector in the null space of A.

$$\boxed{\{\ \vec{x}^* + \vec{x}_0 \mid \vec{x}_0 \in \text{nullsp}(A)\ \}}$$

This set is *not* a subspace, because it does not necessarily pass through the origin (for instance, $\vec{x}^*$ itself may not be $\vec{0}$).

f) (3 pts) There are infinitely many vectors $\vec{x}^*$ that minimize $\|\vec{b} - A\vec{x}\|^2$. If we try and use code to find a solution, it can't return all of them — it'll pick a particular one.
In Python, use `np.linalg.lstsq` to find a vector $\vec{x}^*$ that minimizes $\|\vec{b} - A\vec{x}\|^2$. Include a screenshot of your code and the vector $\vec{x}^*$ it returns, and in your PDF, write out the coefficients of $\vec{x}^*$ as a vector (in addition to the screenshot). Then, provide an educated guess of **why** you think it picked the $\vec{x}^*$ that it did.

Solution:
When fitting a linear model, the goal of `np.linalg.lstsq` is to find a weight vector $\vec{w}^*$ that minimizes

$$\|\vec{b} - A\vec{w}\|^2$$

If A^TA is invertible (that is, if the columns of A are linearly independent), there is exactly one solution:

$$\vec{w}^* = (A^TA)^{-1}A^T\vec{b}$$

However, if A does not have full rank, then A^TA cannot be inverted, and there are infinitely many $\vec{w}$ that give the same minimum error.

In this case, `np.linalg.lstsq` uses the **singular value decomposition (SVD)** of A to find a stable and well-defined $\vec{w}^*$. Without getting into the details, know that among all possible minimizers, this particular $\vec{w}^*$ also has the smallest possible length $\|\vec{w}\|$. In other words, it fits the data as well as possible while keeping the parameter vector as small as possible. It is called the "min-norm" solution.

Problem 4: The Projection Matrix (16 pts)

The projection of a vector $\vec{y} \in \mathbb{R}^n$ onto colsp(X) is the vector $\vec{p} \in$ colsp(X) that is closest to $\vec{y}$. We can write $\vec{p} = X\vec{w}^*$, where $\vec{w}^*$ minimizes $\|\vec{y} - X\vec{w}\|^2$.

Throughout this problem, let X be an $n \times d$ matrix with linearly independent columns. Then X^TX is invertible, and

$$\vec{p} = X(X^TX)^{-1}X^T\vec{y} = P\vec{y}, \qquad P = X(X^TX)^{-1}X^T$$

The $n \times n$ matrix P is called the **projection matrix**, discussed in Chapter 6.4.

This is not a new idea: Homework 5, Problem 5 was about the projection matrix for a 1-dimensional column space. Here, we're looking at the projection matrix for a column space of any dimension!

a) (7 pts) In this part only, suppose $X = \vec{x}$, where $\vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ is a unit vector in $\mathbb{R}^2$. Then,

$$P = \vec{x}\vec{x}^T = \begin{bmatrix} x_1^2 & x_1x_2 \\ x_1x_2 & x_2^2 \end{bmatrix}$$

Find:

(i) rank(P)

(ii) A basis for colsp(P)

(iii) A basis for nullsp(P)

(iv) A basis for colsp(P^T)

b) (4 pts) For the rest of the problem, return to assuming X is an $n \times d$ matrix with linearly independent columns, and

$$P = X(X^TX)^{-1}X^T$$

Show that P is both symmetric (meaning that $P^T = P$) and idempotent (meaning that $P^2 = P$). Then, explain in English how P's idempotence relates to projecting $\vec{y}$ onto colsp(X).

c) (2 pts) Suppose $d < n$. Explain why P is not invertible, and why the transformation $f(\vec{y}) = P\vec{y}$ can't be reversed.

d) (3 pts) Now suppose $d = n$. What is P? What does this say about the relationship between $\vec{y}$, $\vec{p}$, and colsp(X)? *Hint: Use the fact that* $(AB)^{-1} = B^{-1}A^{-1}$.

Problem 5: Same, but Different (11 pts)

In Chapter 2.4, we were introduced to one of many formulas for the optimal slope, w_1^*, and optimal intercept, w_0^*, for the simple linear regression model $h(x_i) = w_0 + w_1 x_i$ when using squared loss:

$$w_1^* = r\frac{\sigma_y}{\sigma_x} \qquad w_0^* = \bar{y} - w_1^* \bar{x}$$

The end goal of Chapters 3 through 6 has been to give us the tools to revisit the simple linear regression model in terms of linear algebra, so that we can extend our model to allow for multiple input variables. As we see in Chapter 7.1, the solution is to define the $n \times 2$ "**design matrix**" X and observation vector $\vec{y} \in \mathbb{R}^n$ as follows:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad \vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Then, the vector containing the optimal model parameters is

$$\vec{w}^* = (X^T X)^{-1} X^T \vec{y} = \begin{bmatrix} w_0^* \\ w_1^* \end{bmatrix}$$

It's not immediately obvious why the components of $\vec{w}^*$ should have anything to do with the correlation, means of x and y, and standard deviations of x and y. In this problem, we will prove that both of these formulations are equivalent, for any dataset (x_1, y_1), (x_2, y_2), ..., (x_n, y_n).

a) (4 pts) Express the matrix $(X^T X)^{-1}$ using constants and/or summations involving x_i and/or y_i.

Solution:

$$X^T X = \begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_n \end{bmatrix} \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} = \begin{bmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{n\sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2} \begin{bmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{bmatrix}$$

b) (3 pts) Prove that

$$(X^T X)^{-1} = \frac{1}{n\sigma_x^2} \begin{bmatrix} \sigma_x^2 + \bar{x}^2 & -\bar{x} \\ -\bar{x} & 1 \end{bmatrix}$$

Hint: Start by proving $\sum_{i=1}^n x_i^2 = n\sigma_x^2 + n\bar{x}^2$.

Solution: From the previous part, we have that

$$(X^TX)^{-1} = \frac{1}{n\sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}\begin{bmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{bmatrix}$$

To simplify, we'll start by proving the statement in the hint, that $\sum_{i=1}^n x_i^2 = n\sigma_x^2 + n\bar{x}^2$. The key "trick" is writing x_i^2 as $(x_i - \bar{x} + \bar{x})^2$, and then expand by grouping $((x_i - \bar{x}) + \bar{x})^2$. We'll use the fact that $\sum_{i=1}^n (x_i - \bar{x}) = 0$; see Chapter 0.1.

$$\begin{aligned}
\sum_{i=1}^n x_i^2 &= \sum_{i=1}^n (x_i - \bar{x} + \bar{x})^2 \\
&= \sum_{i=1}^n ((x_i - \bar{x}) + \bar{x})^2 \\
&= \sum_{i=1}^n \left((x_i - \bar{x})^2 + 2(x_i - \bar{x})\bar{x} + \bar{x}^2\right) \\
&= \sum_{i=1}^n (x_i - \bar{x})^2 + 2\bar{x}\underbrace{\sum_{i=1}^n (x_i - \bar{x})}_{0} + n\bar{x}^2 \\
&= \sum_{i=1}^n (x_i - \bar{x})^2 + n\bar{x}^2 \\
&= n\sigma_x^2 + n\bar{x}^2
\end{aligned}$$

How does this help us? Looking back at

$$(X^TX)^{-1} = \frac{1}{n\sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}\begin{bmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{bmatrix}$$

We can make the following substitutions:

- $n\sum_{i=1}^n x_i^2 = n(n\sigma_x^2 + n\bar{x}^2) = n^2\sigma_x^2 + n^2\bar{x}^2$
- $(\sum_{i=1}^n x_i)^2 = (n\bar{x})^2 = n^2\bar{x}^2$
- $\sum_{i=1}^n x_i^2 = n\sigma_x^2 + n\bar{x}^2$
- $-\sum_{i=1}^n x_i = -n\bar{x}$

So,

$$
\begin{aligned}
(X^TX)^{-1} &= \frac{1}{n\sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i\right)^2}\begin{bmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{bmatrix} \\
&= \frac{1}{n^2\sigma_x^2 + n^2\bar{x}^2 - n^2\bar{x}^2}\begin{bmatrix} n\sigma_x^2 + n\bar{x}^2 & -n\bar{x} \\ -n\bar{x} & n \end{bmatrix} \\
&= \underbrace{\frac{1}{n^2\sigma_x^2}\begin{bmatrix} n\sigma_x^2 + n\bar{x}^2 & -n\bar{x} \\ -n\bar{x} & n \end{bmatrix}}_{\text{divide matrix by } n} \\
&= \frac{1}{n\sigma_x^2}\begin{bmatrix} \sigma_x^2 + \bar{x}^2 & -\bar{x} \\ -\bar{x} & 1 \end{bmatrix}
\end{aligned}
$$

c) (4 pts) Finally, prove that

$$(X^TX)^{-1}X^T\vec{y} = \begin{bmatrix} \bar{y} - r\frac{\sigma_y}{\sigma_x}\bar{x} \\ r\frac{\sigma_y}{\sigma_x} \end{bmatrix}$$

Hint: Start by proving that $\sum_{i=1}^n x_iy_i = nr\sigma_x\sigma_y + n\bar{x}\bar{y}$.

Solution:
We already have a simplified form for $(X^TX)^{-1}$ from the previous part. Let's look at $X^T\vec{y}$.

$$X^T\vec{y} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{bmatrix}$$

This is where the hint comes in. Let's show that $\sum_{i=1}^n x_i y_i = nr\sigma_x\sigma_y + n\bar{x}\bar{y}$. It's not immediately clear how to approach this, since starting with just $\sum_{i=1}^n x_i y_i$ doesn't seem to help us. Instead, let's start by expanding the definition of r.

$$\begin{aligned}
r &= \frac{1}{n}\sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma_x}\right)\left(\frac{y_i - \bar{y}}{\sigma_y}\right) \\
nr\sigma_x\sigma_y &= \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \\
nr\sigma_x\sigma_y &= \sum_{i=1}^n (x_i y_i - x_i\bar{y} - y_i\bar{x} + \bar{x}\bar{y}) \\
nr\sigma_x\sigma_y &= \sum_{i=1}^n x_i y_i - \bar{y}\sum_{i=1}^n x_i - \bar{x}\sum_{i=1}^n y_i + \sum_{i=1}^n \bar{x}\bar{y} \\
nr\sigma_x\sigma_y &= \sum_{i=1}^n x_i y_i - n\bar{y}\bar{x} - n\bar{x}\bar{y} + n\bar{x}\bar{y} \\
nr\sigma_x\sigma_y &= \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} \\
\sum_{i=1}^n x_i y_i &= nr\sigma_x\sigma_y + n\bar{x}\bar{y}
\end{aligned}$$

So, that gives us

$$X^T\vec{y} = \begin{bmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{bmatrix} = \begin{bmatrix} n\bar{y} \\ n(r\sigma_x\sigma_y + \bar{x}\bar{y}) \end{bmatrix}$$

Finally, we're ready to evaluate $\vec{w}^* = (X^TX)^{-1}X^T\vec{y}$.

$$
\begin{aligned}
(X^TX)^{-1}X^T\vec{y} &= \frac{1}{n\sigma_x^2}\begin{bmatrix}\bar{x}^2+\sigma_x^2 & -\bar{x}\\ -\bar{x} & 1\end{bmatrix}\begin{bmatrix}n\bar{y}\\ n(r\sigma_x\sigma_y+\bar{x}\bar{y})\end{bmatrix}\\
&= \frac{1}{\sigma_x^2}\begin{bmatrix}\bar{x}^2+\sigma_x^2 & -\bar{x}\\ -\bar{x} & 1\end{bmatrix}\begin{bmatrix}\bar{y}\\ r\sigma_x\sigma_y+\bar{x}\bar{y}\end{bmatrix}\\
&= \frac{1}{\sigma_x^2}\begin{bmatrix}(\bar{x}^2+\sigma_x^2)\bar{y}-\bar{x}(r\sigma_x\sigma_y+\bar{x}\bar{y})\\ -\bar{x}\bar{y}+r\sigma_x\sigma_y+\bar{x}\bar{y}\end{bmatrix}\\
&= \frac{1}{\sigma_x^2}\begin{bmatrix}\bar{x}^2\bar{y}+\sigma_x^2\bar{y}-\bar{x}r\sigma_x\sigma_y-\bar{x}^2\bar{y}\\ r\sigma_x\sigma_y\end{bmatrix}\\
&= \frac{1}{\sigma_x^2}\begin{bmatrix}\sigma_x^2\bar{y}-\bar{x}r\sigma_x\sigma_y\\ r\sigma_x\sigma_y\end{bmatrix}\\
&= \begin{bmatrix}\bar{y}-r\frac{\sigma_y}{\sigma_x}\bar{x}\\ r\frac{\sigma_y}{\sigma_x}\end{bmatrix}
\end{aligned}
$$

Finally!

Note that the second component of the vector above is $w_1^* = r\frac{\sigma_y}{\sigma_x}$ and the first component of the vector above is $w_0^* = \bar{y} - r\frac{\sigma_y}{\sigma_x}\bar{x} = \bar{y} - w_1^*\bar{x}$, as we first saw in Chapter 2.4! I think this is beautiful.

Problem 6: Putting it into Practice (8 pts)

This problem asks you to apply the concepts in Chapter 7.2, and follows the last problem.

Suppose we'd like to fit a hypothesis function of the form $h(x_i) = w_0 + w_1x_i^2$. Notice the squared term; this is **not** a simple linear regression line.

To do so, we'll find the optimal parameter vector $\vec{w}^*$ that satisfies the normal equations. The first 5 rows of our dataset are as follows, though note that our dataset has n rows in total.

x_i	y_i
2	4
-1	2
3	5
-7	3
3	-7
$\vdots$	$\vdots$

Suppose $x_1, x_2, ..., x_n$ have a mean of $\bar{x} = 5$ and a variance of $\sigma_x^2 = 8$.

a) (2 pts) Write the first five rows of the design matrix, X.

Solution: Since our hypothesis function is $h(x_i) = w_0 + w_1 x_i^2$, the design matrix must include a column of ones (for the intercept term) and a column containing x_i^2 for each x_i value.

$$X = \begin{bmatrix} 1 & x_1^2 \\ 1 & x_2^2 \\ 1 & x_3^2 \\ 1 & x_4^2 \\ 1 & x_5^2 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 1 & 1 \\ 1 & 9 \\ 1 & 49 \\ 1 & 9 \end{bmatrix}$$

Here, each entry in the second column corresponds to x_i^2 for the given x_i values in the dataset.

b) (3 pts) Suppose that after solving the normal equations, we find $\vec{w}^* = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$. Find the augmented feature vector $\text{Aug}(\vec{x}_3)$ and squared loss for row 3 of the dataset.

Solution: The augmented feature vector adds a constant 1 for the intercept and the squared value of x_3:

$$\text{Aug}(\vec{x}_3) = \begin{bmatrix} 1 \\ x_3^2 \end{bmatrix} = \begin{bmatrix} 1 \\ 9 \end{bmatrix}$$

The hypothesis function is

$$h(x_i) = w_0 + w_1 x_i^2$$

Plugging in $x_3 = 3$ and $\vec{w}^* = [5, -2]^T$:

$$h(x_3) = 5 + (-2)(9) = -13$$

From the dataset, $y_3 = 5$. The error for this data point is:

$$e_3 = y_3 - h(x_3) = 5 - (-13) = 18$$

The squared loss is

$$L_{\text{sq}}(y_3, h(x_3)) = (y_3 - h(x_3))^2 = (18)^2 = 324$$

$$\boxed{\text{Aug}(\vec{x}_3) = \begin{bmatrix} 1 \\ 9 \end{bmatrix} \quad L_{\text{sq}}(y_3, h(x_3)) = 324}$$

c) (3 pts) Let $X_{\text{tri}} = 3X$, where X is the full design matrix for our dataset with n rows. Determine the bottom-left value in the matrix $X_{\text{tri}}^T X_{\text{tri}}$, i.e. the value in the second row and first column. Your answer should be an expression involving n. *Hint: You can use any of the hints or results from Problem 5 without needing to re-prove them.*

Solution: Starting with the design matrix

$$X = \begin{bmatrix} 1 & x_1^2 \\ 1 & x_2^2 \\ \vdots & \vdots \\ 1 & x_n^2 \end{bmatrix} \qquad X_{\text{tri}} = 3X = \begin{bmatrix} 3 & 3x_1^2 \\ 3 & 3x_2^2 \\ \vdots & \vdots \\ 3 & 3x_n^2 \end{bmatrix}$$

The matrix product is

$$X_{\text{tri}}^T X_{\text{tri}} = \begin{bmatrix} 3 & 3 & \cdots & 3 \\ 3x_1^2 & 3x_2^2 & \cdots & 3x_n^2 \end{bmatrix} \begin{bmatrix} 3 & 3x_1^2 \\ 3 & 3x_2^2 \\ \vdots & \vdots \\ 3 & 3x_n^2 \end{bmatrix}$$

The bottom-left entry (row 2, column 1) is

$$\sum_{i=1}^{n} (3x_i^2)(3) = \sum_{i=1}^{n} 9x_i^2 = 9\sum_{i=1}^{n} x_i^2.$$

Using the identity $\sum_{i=1}^{n} x_i^2 = n\sigma_x^2 + n\bar{x}^2$,

$$\sum_{i=1}^{n} x_i^2 = n\sigma_x^2 + n\bar{x}^2$$

we substitute to obtain

$$\text{Bottom-left entry} = 9n(\sigma_x^2 + \bar{x}^2)$$

and finally, substituting $\sigma_x^2 = 8$ and $\bar{x} = 5$:

$$9n(8 + 5^2) = 9n(8 + 25) = 9n(33) = \boxed{297n}$$

Problem 7: Orthogonalization (22 pts)

Before starting, refer to Chapter 6.5, written just for this problem. It won't be possible to do this problem without referencing it.

In parts **a)** through **d)**, we'll refer to the vectors $\vec{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $\vec{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$, and $\vec{v}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$.

a) (5 pts) By hand, apply the Gram-Schmidt process to the vectors $\vec{v}_1, \vec{v}_2, \vec{v}_3$ to find an orthonormal set of vectors $\vec{q}_1, \vec{q}_2, \vec{q}_3$. Show your work; you cannot use `numpy` for this.

Then, create the matrix $Q = \begin{bmatrix} | & | & | \\ \vec{q}_1 & \vec{q}_2 & \vec{q}_3 \\ | & | & | \end{bmatrix}$ and confirm that $Q^T Q = I$, but that $QQ^T \neq I$.

Solution:

- **Iteration 1**: Set $\vec{Q}_1 = \vec{v}_1 = \begin{bmatrix}1\\1\\0\\0\end{bmatrix}$.
- **Iteration 2**: Set $\vec{Q}_2 = \vec{v}_2 - \text{proj}_{\vec{Q}_1}(\vec{v}_2) = \vec{v}_2 - \frac{\vec{v}_2\cdot\vec{Q}_1}{\vec{Q}_1\cdot\vec{Q}_1}\vec{Q}_1 = \begin{bmatrix}0\\1\\1\\0\end{bmatrix} - \frac{1}{2}\begin{bmatrix}1\\1\\0\\0\end{bmatrix} = \begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}$.
- **Iteration 3**: Set $\vec{Q}_3 = \vec{v}_3 - \text{proj}_{\vec{Q}_1}(\vec{v}_3) - \text{proj}_{\vec{Q}_2}(\vec{v}_3)$:

$$\begin{aligned}
\vec{Q}_3 &= \vec{v}_3 - \text{proj}_{\vec{Q}_1}(\vec{v}_3) - \text{proj}_{\vec{Q}_2}(\vec{v}_3)\\
&= \begin{bmatrix}0\\0\\1\\1\end{bmatrix} - \underbrace{\frac{\vec{v}_3\cdot\vec{Q}_1}{\vec{Q}_1\cdot\vec{Q}_1}\vec{Q}_1}_{\vec{v}_3\cdot\vec{Q}_1=0} - \frac{\vec{v}_3\cdot\vec{Q}_2}{\vec{Q}_2\cdot\vec{Q}_2}\vec{Q}_2\\
&= \begin{bmatrix}0\\0\\1\\1\end{bmatrix} - \frac{\vec{v}_3\cdot\vec{Q}_2}{\vec{Q}_2\cdot\vec{Q}_2}\vec{Q}_2\\
&= \begin{bmatrix}0\\0\\1\\1\end{bmatrix} - \frac{\begin{bmatrix}0\\0\\1\\1\end{bmatrix}\cdot\begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}}{\begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}\cdot\begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}}\begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}\\
&= \begin{bmatrix}0\\0\\1\\1\end{bmatrix} - \frac{2}{3}\begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix} = \begin{bmatrix}1/3\\-1/3\\1/3\\1\end{bmatrix}
\end{aligned}$$

So,

$$\vec{Q}_1 = \begin{bmatrix}1\\1\\0\\0\end{bmatrix}, \quad \vec{Q}_2 = \begin{bmatrix}-1/2\\1/2\\1\\0\end{bmatrix}, \quad \vec{Q}_3 = \begin{bmatrix}1/3\\-1/3\\1/3\\1\end{bmatrix}$$

Finally, we need to normalize the vectors to have length 1. Doing so gives

$$\boxed{\vec{q}_1 = \frac{\vec{Q}_1}{\|\vec{Q}_1\|} = \begin{bmatrix}1/\sqrt{2}\\1/\sqrt{2}\\0\\0\end{bmatrix}, \quad \vec{q}_2 = \frac{\vec{Q}_2}{\|\vec{Q}_2\|} = \begin{bmatrix}-1/\sqrt{6}\\1/\sqrt{6}\\2/\sqrt{6}\\0\end{bmatrix}, \quad \vec{q}_3 = \frac{\vec{Q}_3}{\|\vec{Q}_3\|} = \begin{bmatrix}\sqrt{3}/6\\-\sqrt{3}/6\\\sqrt{3}/6\\\sqrt{3}/2\end{bmatrix}}$$

(Tip: Instead of converting $\vec{Q}_3 = \begin{bmatrix} 1/3 \\ -1/3 \\ 1/3 \\ 1 \end{bmatrix}$ to a unit vector, it's easier to instead convert $\begin{bmatrix} 1 \\ -1 \\ 1 \\ 3 \end{bmatrix}$, which points in the same direction as $\vec{Q}_3$, to a unit vector. The result will be the same in both cases, but by multiplying through by 3 you get to avoid messier fractions.)

Finally, we need to construct $Q = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{6} & \sqrt{3}/6 \\ 1/\sqrt{2} & 1/\sqrt{6} & -\sqrt{3}/6 \\ 0 & 2/\sqrt{6} & \sqrt{3}/6 \\ 0 & 0 & \sqrt{3}/2 \end{bmatrix}$. Indeed, $Q^TQ = I$ because all pairs of columns are orthogonal and have length 1. However, $QQ^T \neq I$ because, for instance, row 3 and row 4 are not orthogonal, so $(QQ^T)_{3,4} = 3/12 = 1/4 \neq 0$.

b) (2 pts) Suppose $\vec{v}_4 = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 3 \end{bmatrix}$. If you were to apply the Gram-Schmidt process to the vectors $\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$, what would the vector $\vec{Q}_4$ be? Why?

Solution: Note that $\vec{v}_4 = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 3 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + 3\begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} = 2\vec{v}_1 + 3\vec{v}_3$, meaning $\{\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4\}$ are linearly dependent.

In the previous part, we already applied Gram-Schmidt to the vectors $\vec{v}_1, \vec{v}_2, \vec{v}_3$, giving us the orthonormal vectors $\vec{q}_1, \vec{q}_2, \vec{q}_3$. If we were to continue the Gram-Schmidt process for a fourth iteration, we'd have

$$\vec{Q}_4 = \vec{v}_4 - \text{proj}_{\vec{Q}_1}(\vec{v}_4) - \text{proj}_{\vec{Q}_2}(\vec{v}_4) - \text{proj}_{\vec{Q}_3}(\vec{v}_4)$$

Gram-Schmidt already gave us $Q = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{6} & \sqrt{3}/6 \\ 1/\sqrt{2} & 1/\sqrt{6} & -\sqrt{3}/6 \\ 0 & 2/\sqrt{6} & \sqrt{3}/6 \\ 0 & 0 & \sqrt{3}/2 \end{bmatrix}$, which has a column space that is equal to the span of $\vec{v}_1, \vec{v}_2, \vec{v}_3$.

c) (3 pts) Consider $\vec{y} = \begin{bmatrix} 3 \\ 1 \\ 2 \\ 4 \end{bmatrix}$. $\vec{y}$ **is** in $\text{span}(\{\vec{v}_1, \vec{v}_2, \vec{v}_3\}) = \text{span}(\{\vec{q}_1, \vec{q}_2, \vec{q}_3\})$.

Find scalars a, b, and c such that $a\vec{q}_1 + b\vec{q}_2 + c\vec{q}_3 = \vec{y}$, **without** solving a system of 3 equations

and 3 unknowns. Instead, use the fact that $\vec{q}_1, \vec{q}_2, \vec{q}_3$ are orthonormal. *Hint: There's a relevant problem from Lab 5.*

Solution: The main idea here is that to write $\vec{y}$ as a linear combination of $\vec{q}_1$, $\vec{q}_2$, and $\vec{q}_3$, we can project $\vec{y}$ onto each of these vectors **individually** and then use the corresponding coefficients to write $\vec{y}$ as a linear combination of all three.

Remember, $\text{proj}_{\vec{q}_i}(\vec{y}) = \frac{\vec{y}\cdot\vec{q}_i}{\vec{q}_i\cdot\vec{q}_i}\vec{q}_i$, but since each $\vec{q}_i$ is a unit vector, we have $\text{proj}_{\vec{q}_i}(\vec{y}) = (\vec{y}\cdot\vec{q}_i)\vec{q}_i$. So,

$$\vec{y} = \text{proj}_{\vec{q}_1}(\vec{y}) + \text{proj}_{\vec{q}_2}(\vec{y}) + \text{proj}_{\vec{q}_3}(\vec{y})$$

$$= \underbrace{(\vec{y}\cdot\vec{q}_1)}_{a}\vec{q}_1 + \underbrace{(\vec{y}\cdot\vec{q}_2)}_{b}\vec{q}_2 + \underbrace{(\vec{y}\cdot\vec{q}_3)}_{c}\vec{q}_3 = a\vec{q}_1 + b\vec{q}_2 + c\vec{q}_3$$

If this idea seems like it's coming out of nowhere, review the relevant projection activity from Lab 4.
So,

$$a = \vec{y}\cdot\vec{q}_1 = \begin{bmatrix}3\\1\\2\\4\end{bmatrix}\cdot\begin{bmatrix}1/\sqrt{2}\\1/\sqrt{2}\\0\\0\end{bmatrix} = 4/\sqrt{2} = \boxed{2\sqrt{2}}$$

$$b = \vec{y}\cdot\vec{q}_2 = \begin{bmatrix}3\\1\\2\\4\end{bmatrix}\cdot\begin{bmatrix}-1/\sqrt{6}\\1/\sqrt{6}\\2/\sqrt{6}\\0\end{bmatrix} = \boxed{2/\sqrt{6}}$$

$$c = \vec{y}\cdot\vec{q}_3 = \begin{bmatrix}3\\1\\2\\4\end{bmatrix}\cdot\begin{bmatrix}\sqrt{3}/6\\-\sqrt{3}/6\\\sqrt{3}/6\\\sqrt{3}/2\end{bmatrix} = \boxed{8\sqrt{3}/3}$$

Notice that $Q^T\vec{y}$ is a vector containing a, b, and c, since $Q^T\vec{y}$ contains the dot product of each of Q^T's rows (which are Q's columns) with $\vec{y}$.

Let's test out our logic in Python.

```
In [1]:
import numpy as np

Q = np.array([
    [1/np.sqrt(2), -1/np.sqrt(6), np.sqrt(3)/6],
    [1/np.sqrt(2), 1/np.sqrt(6), -np.sqrt(3)/6],
    [0           , 2/np.sqrt(6),  np.sqrt(3)/6],
    [0           , 0           ,  np.sqrt(3)/2]])

In [3]:
Q.T @ np.array([3, -1, -2, -2])

Out[3]:
array([ 1.41421356, -3.26598632, -1.15470054])

In [4]:
# a - same as first component above
np.sqrt(2)

Out[4]:
1.4142135623730951

In [5]:
# b - same as second component above
-8/np.sqrt(6)

Out[5]:
-3.2659863237109046

In [6]:
# c - same as third component above
-2 * np.sqrt(3)/3

Out[6]:
-1.1547005383792515
```

d) (3 pts) Consider $\vec{y} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$. Unlike in **c)**, $\vec{y}$ **is not** in span$(\{\vec{v}_1, \vec{v}_2, \vec{v}_3\})$.

Find the vector in span$(\{\vec{v}_1, \vec{v}_2, \vec{v}_3\})$ that is closest to $\vec{y}$. **Do not** stack the $\vec{v}_i$'s into a matrix X and then use $X(X^TX)^{-1}X^T\vec{y}$. Instead, use the fact that $\vec{q}_1, \vec{q}_2, \vec{q}_3$ are orthonormal and have the same span as $\vec{v}_1, \vec{v}_2, \vec{v}_3$. How does this simplify the problem?

Solution: Recall, the projection of $\vec{y}$ onto span$(\{\vec{q}_1, \vec{q}_2, \vec{q}_3\})$ is given by

$$\vec{p} = \underbrace{Q(Q^TQ)^{-1}Q^T}_{P}\vec{y}$$

Since $Q^TQ = I$, we have that

$$\vec{p} = QQ^T\vec{y}$$

If Q's rows were orthonormal, then $QQ^T = I$, but that's not the case here. Instead, $\vec{p} = QQ^T\vec{y}$.

$$QQ^T = \begin{bmatrix} 3/4 & 1/4 & -1/4 & 1/4 \\ 1/4 & 3/4 & 1/4 & -1/4 \\ -1/4 & 1/4 & 3/4 & 1/4 \\ 1/4 & -1/4 & 1/4 & 3/4 \end{bmatrix}$$

$$\implies \vec{p} = QQ^T\vec{y} = \begin{bmatrix} 3/4 & 1/4 & -1/4 & 1/4 \\ 1/4 & 3/4 & 1/4 & -1/4 \\ -1/4 & 1/4 & 3/4 & 1/4 \\ 1/4 & -1/4 & 1/4 & 3/4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3/2 \\ 3/2 \\ 7/2 \\ 7/2 \end{bmatrix}$$

So, the vector in span$(\{\vec{v}_1, \vec{v}_2, \vec{v}_3\})$ that is closest to $\vec{y}$ is $\boxed{\begin{bmatrix} 3/2 \\ 3/2 \\ 7/2 \\ 7/2 \end{bmatrix}}$.

e) (4 pts) Open the **the supplemental Jupyter Notebook** we've created for Homework 7, which can either be found **here** in the course GitHub repository, or **here** on DataHub.

There, you're asked to implement the function `orthogonalize`, which takes in an $n \times d$ matrix V whose columns are linearly independent, and returns a matrix Q whose columns are orthonormal and have the same span as V. This problem is **not autograded**. Rather, in your submission to this part, include a screenshot of your implementation and sample output in your PDF for Homework 7.

Solution:

In [82]:
```
# BEGIN SOLUTION NO PROMPT
def project(y, x):
    """
    Assumes y and x are column vectors.
    Returns the projection of y onto x.
    This is a vector in the direction of x.
    """
    return x * np.dot(y, x) / np.dot(x, x)
# END SOLUTION
def orthogonalize(V):
    """
    Runs the Gram-Schmidt process on the columns of V.
    The shape of Q should be the same as the shape of V.
    Assume V has at least 1 column and 1 row.
    """
    V = V.copy() # Ensures that you don't make any modifications to the underlying array.
    # BEGIN SOLUTION
    Q = np.zeros_like(V)

    for j in range(V.shape[1]):
        v_j = V[:, j]
        Q_j = v_j
        for i in range(j):
            Q_j -= project(v_j, Q[:, i])
        Q[:, j] = Q_j / np.linalg.norm(Q_j)
    return Q
    # END SOLUTION

# Feel free to test out your implementation below.
# Recent versions of numpy will error if you try and perform operations involving
# an array of ints and an array of floats, which is why we've initialized the array with floats.
orthogonalize(np.array([[4.0, 1.0],
                        [3.0, 5.0]]))
```

Out[82]:
```
array([[ 0.8, -0.6],
       [ 0.6,  0.8]])
```

In [83]:
```
# DO NOT CHANGE THIS CELL;
# include a screenshot of the output of this cell in your submitted PDF.
V_test_1 = np.array([[1.0,  1.0, 1.0],
                     [-1.0, 0.0, 1.0],
                     [1.0,  1.0, 2.0]])
Q_test_1 = orthogonalize(V_test_1)
Q_test_1
```

Out[83]:
```
array([[ 5.77350269e-01,  4.08248290e-01, -7.07106781e-01],
       [-5.77350269e-01,  8.16496581e-01, -3.14018492e-16],
       [ 5.77350269e-01,  4.08248290e-01,  7.07106781e-01]])
```

In [84]:
```
# DO NOT CHANGE THIS CELL;
# include a screenshot of the output of this cell in your submitted PDF.
V_test_2 = np.array([[3, 5, 4, 0],
                     [1, 0, 9, -1],
                     [5, 1, 3, 2],
                     [0, 8, 0, 8],
                     [3, 2, -1.5, 4],
                     [0, 0, 0, 1]])
Q_test_2 = orthogonalize(V_test_2)
Q_test_2
```

Out[84]:
```
array([[ 0.45226702,  0.36393511,  0.19396785, -0.76063409],
       [ 0.15075567, -0.06663601,  0.89611774,  0.31406129],
       [ 0.75377836, -0.22041141, -0.05888979,  0.21816459],
       [ 0.        ,  0.90214901, -0.01523765,  0.3750409 ],
       [ 0.45226702,  0.02562923, -0.39452412,  0.29233934],
       [ 0.        ,  0.        ,  0.        ,  0.22155708]])
```

f) (4 pts) A QR decomposition of a matrix A is a factorization of the form

$$A = QR$$

where Q is an $n \times d$ matrix with orthonormal columns and R is an $d \times d$ **upper triangular** matrix (a matrix that has 0s below the diagonal).

For example, if $A = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 1 & 2 \end{bmatrix}$, a QR decomposition of A is

$$A = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 1 & 2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1/\sqrt{3} & 1/\sqrt{6} & -1/\sqrt{2} \\ -1/\sqrt{3} & 2/\sqrt{6} & 0 \\ 1/\sqrt{3} & 1/\sqrt{6} & 1/\sqrt{2} \end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} \sqrt{3} & 2\sqrt{3}/3 & 2\sqrt{3}/3 \\ 0 & \sqrt{6}/3 & 5\sqrt{6}/6 \\ 0 & 0 & 1/\sqrt{2} \end{bmatrix}}_{R}$$

Finding the Q in a QR decomposition is straightforward: apply Gram-Schmidt to the columns of A, assuming A's columns are linearly independent. The question is how to find R.

(i) In the supplemental Jupyter Notebook, we've defined an arbitrary matrix A and call your `orthogonalize` function on it, and give you hints as to how to find R. Using the experimentation there, and what you know about Q, **explain how to find R.**

(ii) Find a QR decomposition of $A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$. Note that the columns of A are made up of the same three vectors you worked with in parts **a)** through **d)** of this problem.

(iii) Given **a** QR decomposition of A, explain how to find **another** QR decomposition of A with a (slightly) different Q and/or R.

Solution:

(i) The big idea is that if Q is a matrix with orthonormal columns, then $Q^TQ = I$. So, if we're trying to find an R such that $A = QR$, then multiplying both sides by Q^T on the left gives us

$$A = QR \implies Q^TA = Q^TQR \implies Q^TA = R$$

meaning that $R = Q^TA$.

All that's left to explain is **why R is upper triangular**. The product Q^TA contains the dot products of the rows of Q^T with the columns of A. But, the rows of Q^T are the columns of Q, so $R = Q^TA$ contains the dot products of columns of Q with columns of A. Specifically,

$$R_{i,j} = \vec{q}_i \cdot \vec{v}_j$$

where $\vec{q}_i$ is the i-th column of Q and $\vec{v}_j$ is the j-th column of A.

Remember, we constructed each $\vec{q}_i$ to be orthogonal to all previously constructed $\vec{q}_j$'s for $j < i$. Put in English, $\vec{q}_2$ is orthogonal to $\vec{q}_1$, $\vec{q}_3$ is orthogonal to $\vec{q}_1$ and $\vec{q}_2$, and so on.

Each $\vec{q}_i$ was found by taking $\vec{v}_i$ and subtracting off **a linear combination of** $\vec{q}_{i-1}, \vec{q}_{i-2}, \ldots, \vec{q}_1$. (More precisely, we built the $\vec{Q}_i$'s this way and then normalized them to get the $\vec{q}_i$'s, but the directions of the $\vec{Q}_i$'s are the same as the directions of the $\vec{q}_i$'s, so this reasoning still holds.) As an example, consider how we would construct $\vec{q}_4$ if we were to apply Gram-Schmidt to the vectors $\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$:

$$\begin{aligned}
\vec{q}_4 &= \vec{v}_4 - \text{proj}_{\vec{q}_1}(\vec{v}_4) - \text{proj}_{\vec{q}_2}(\vec{v}_4) - \text{proj}_{\vec{q}_3}(\vec{v}_4) \\
\vec{q}_4 &= \vec{v}_4 - \frac{\vec{v}_4 \cdot \vec{q}_1}{\vec{q}_1 \cdot \vec{q}_1}\vec{q}_1 - \frac{\vec{v}_4 \cdot \vec{q}_2}{\vec{q}_2 \cdot \vec{q}_2}\vec{q}_2 - \frac{\vec{v}_4 \cdot \vec{q}_3}{\vec{q}_3 \cdot \vec{q}_3}\vec{q}_3 \\
\vec{q}_4 &= \vec{v}_4 - a\vec{q}_1 - b\vec{q}_2 - c\vec{q}_3 \\
\implies a\vec{q}_1 + b\vec{q}_2 + c\vec{q}_3 + \vec{q}_4 &= \vec{v}_4
\end{aligned}$$

But, since all the $\vec{q}_i$'s are orthogonal to each other, if we were to take the dot product of both sides of the equation with $\vec{q}_5$, or $\vec{q}_6$, or any other $\vec{q}_i$ for $i > 4$, we would get 0.

This illustrates that $\vec{q}_i \cdot \vec{v}_j = 0$ when $i > j$. But, a matrix with 0's where $i > j$ is a matrix with 0's everywhere below the diagonal of $i = j$, which is precisely an upper triangular matrix.

$$\underbrace{\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & 0 & \cdot & \cdot \\ 0 & 0 & 0 & \cdot \end{bmatrix}}_{\text{in all the 0's, the row index (i) is greater than the column index (j)}}$$

So, since $\vec{q}_i \cdot \vec{v}_j = 0$ when $i > j$, $R = Q^T A$ — which is made up of these dot products — is upper triangular.

(ii) We have already found the Q in a QR decomposition of $A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$: these are the vectors we found in part **a)** of the problem.

$$Q = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{6} & \sqrt{3}/6 \\ 1/\sqrt{2} & 1/\sqrt{6} & -\sqrt{3}/6 \\ 0 & 2/\sqrt{6} & \sqrt{3}/6 \\ 0 & 0 & \sqrt{3}/2 \end{bmatrix}$$

Then, $R = Q^T A$ is

$$R = Q^T A = \underbrace{\begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 & 0 \\ -1/\sqrt{6} & 1/\sqrt{6} & 2/\sqrt{6} & 0 \\ \sqrt{3}/6 & -\sqrt{3}/6 & \sqrt{3}/6 & 3\sqrt{3}/6 \end{bmatrix}}_{Q^T} \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 3/\sqrt{6} & 2/\sqrt{6} \\ 0 & 0 & 2\sqrt{3}/3 \end{bmatrix}$$

(iii) An easy solution is to negate one or more of the columns of Q; the resulting columns of Q will still be orthonormal with the same column space. This will lead to a slightly different R. (There are other ways of finding a QR decomposition as well.)

g) (1 pt) Suppose we replace the design matrix X with a matrix Q whose columns are orthonormal. How does the normal equation $X^T X \vec{w}^* = X^T \vec{y}$ simplify? Give an expression for $\vec{w}^*$.

Solution: Since $Q^T Q = I$, the normal equation becomes $\vec{w}^* = Q^T \vec{y}$.

Problem 8: Billy the Waiter (13 pts)

This problem involves writing code and submitting it to the Pensive autograder. The goal of this problem is to give you a taste of how linear algebra can be used to implement linear regression in code, and show you how to build models that involve multiple features (including categorical variables).

Open the **the supplemental Jupyter Notebook** we've created for Homework 7, which can either be found **here** in the course GitHub repository, or **here** on DataHub.

This problem is entirely autograded; to receive credit for Problem 8 of this homework, you'll need to submit your completed notebook to the autograder on Pensive. Your submission time for Homework 7 is the **latest** of your video link, PDF, and code submission times.